Copyright of Algonquin College Faculty, do not distribute

# CST8116 Assignment 01 (24F)

Software Development Process, design and create an Object-Oriented Java Program

## Instructions and Important Notes

A rubric is provided for this lab assignment. Review your work before submitting. In addition, the following deductions will apply:

- If you use any tool to generate java code or UML, you will be given zero for the assignment. This includes a tool which converts UML to Java or vice versa as well as AI tools.
- If your solution differs from the normal output of the tools we allow you to use, then you will get zero for the assignment even if that solution is correct. The allowed tools are Eclipse, UMLet, draw.io, diagrams.net and MS Word.
- Submissions without .java source code files will receive a maximum grade of 20%
- Submissions that are not object oriented (all input/output/computations are in main and there is no separate object class for the pyramid) will receive a maximum grade of 30%
- Submissions that do not use the Java API Math class to both computing a value using an exponent and for obtaining the value of PI will have 20% grade deduction (or 10% for each).
- Submissions not using printf to format the result to 3 decimal places will have a 5% grade deduction.

The Software Development Process by Cay Horstmann [1] will be used as the basis for this lab assignment.

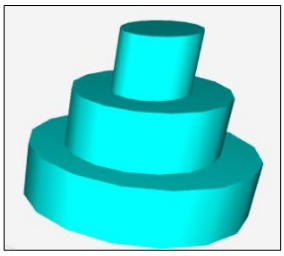
1) Understand the problem
2) Develop and Describe an Algorithm
3) Test Algorithm with Simple Inputs
4) Translate the Algorithm into Java
5) Compile and Test Your Program

## Learning Resources

Week 2 Lecture regarding classes Math, Scanner
Week 3 Lecture materials regarding creating a class, with variables, constructors, get & set methods, worker method(s)
Hybrid 3 for more on UML Class diagrams and using the UMLet software.
Hybrid 4 for information on using printf to format numeric output.
See [1] for more help with Java, and [2] for more help with pseudocode and UML Class Diagrams

## What is the problem to solve?

A cylinder pyramid is created when you stack multiple right-circular cylinders together, each is smaller than the one below, as seen in the figure on the right.



- Create an Object-Oriented Java program which calculates the volume of a cylinder pyramid with **3** cylinders, given the bottom cylinder's dimensions.
- **NOTE**: Assume that each cylinder's radius is smaller than of the cylinder below it by 20% (but keeping the same cylinder height).
- Minimally, you will create two classes, one to represent or model a cylinder pyramid, and another with a main method.
- The main method will get user input, create a cylinder pyramid object, and make method calls on that object. The class that models the cylinder pyramid must perform the volume calculation using a worker method and return its value to the main method, which will output the volume formatted to **3** decimal places.
- The program will also output the programmer's name. (See sample output in the appendix)
- A sample resource for a single cylinder's volume can be seen here: https://www.omnicalculator.com/math/cylinder-volume

**NOTE**: Fill the provided submission template to complete steps 1, 2, 3, 5 below.

## 1) Understand the problem

- Locate at least one website OTHER THAN THE ONE GIVEN, that details the formula needed to calculate the volume of a cylinder. Write the formula into your MS Word document, on one line as plain-text, and cite and reference your new source in IEEE reference format.
  (Tip: See lecture notes week 1 for a document to help with citations and references).
- Write a formula that will perform the cylinder pyramid volume calculation given the problem's requirements. It must use exponent math, as in: **$E = mc^2$**
- Write 3 sample calculations using your own values. (For example, calculate the volume when the bottom cylinder's radius is 0.5 and height is 2)

## 2) Develop and Describe an Algorithm including Classes (MS-Word Document Submission)

### 2a) UML Class Diagrams

- Determine what objects, the properties of the objects, and behaviors would be required for an object-oriented program, start with simple UML class diagrams and as you work out more details document the design with a detailed UML class diagram.
- Include your detailed UML Class diagrams in your MS Word document. (See lecture notes week 3).
- One class diagram should document field variables, constructor(s), getter(s), setter(s), and one worker method.
- One class diagram should document a class that only contains method main. The class name should be Driver.

### 2b) Pseudocode (MS-Word Document Submission)

- NOTE: You are expected to use the Java API Math class to obtain a value for PI, and to calculate the power of the radius (or diameter) raised to 2.
- Write pseudocode for method main, as well as a method(s) to calculate the cylinder pyramid's volume.
- Include your pseudocode in your MS Word document.
- Note: Do not write pseudocode for the get or set methods or constructors for this assignment, the expectation would be method main and one worker methods.

## 3) Algorithm Test Table (MS-Word Document Submission)

- As per the lecture notes, use a table within your MS Word document to test the algorithm for method main. Use the 3 values selected in step 1 (Understand the problem) and work through the algorithm documenting expected outputs.
- You must have 3 rows (tests) in your table!
- If there is a problem with the algorithm based on this desk-check (using the test plan), correct the pseudocode and UML class diagram(s) and repeat this step again.

## 4) Translate the Algorithm into Java

- You are to use the Eclipse IDE to create your Java program, use a project name like Assignment 01.
- Don't forget to comment your code files, with the expected code headers
  - File-level comments, similar to an assignment cover page.
  - Class-header comment, just above the class declaration with brief description of the class
  - Method-header comment(s), just above each method or constructor with brief description
  - Cite and Reference the web source where you located the formula for the volume of a cylinder.

## 5) Java Program Test Table (MS-Word Document Submission)

- Re-create your testing table from step 3 in this section, but document whether the program outputs are using valid inputs and does the output match expectations.
- Test with some invalid inputs, and document what happens. Note that some input will crash your program, at this point of in the course this is okay as we didn't cover how to fix this yet. Document what the error messages are in your test plan. Suggested invalid tests: enter a String instead of a number, use a negative number as input, use zero as part of the input. Try some things and document what happens.
- You must have the original 3 rows (tests) in your table, plus 3 additional rows of invalid inputs!
- **The test plan for the program must be a separate table in your document, if you provide only one test plan table for both testing the algorithm and testing the program you will not be awarded marks for both. Minimally, this second table must also have each row updated to indicate that it was used to test the program, in other words if you copy and paste an exact copy of your first table unchanged you will not earn full marks.**

```
public class PyramidCylinder {

 private double radius;
 private double height;

 public PyramidCylinder(double radius,
double height) {
  this.radius = radius;
  this.height = height;
 }

 public double getRadius() {
  return radius;
 }
}
```

## Microsoft Word Document: Suggested Headings

Understand the problem
UML Class Diagram(s)
Pseudocode
Algorithm Test Table
Program Test Table
References

## Submission Requirements

- You will need to submit your MS Word document and your Java source code file(s)
- Follow your lab professor's submission guidelines.
- **Do not** copy and paste Java code into the MS Word document. Your .java file(s) must be submitted as Java source code files alongside your MS Word document.
- Verify your submission based on the requirements listed in this document and the rubric below

## Rubric

Available on Brightspace under **Activities** -> **Rubrics**

```
import java.util.Scanner;

public class Main {
 public static void main(String[] args) {

  double radius;
  double height;

  Scanner keyboard = new Scanner
(System.in);

  System.out.println("please enter
radius");
  radius = keyboard.nextDouble();

  System.out.println("please enter
height");
  height = keyboard.nextDouble();
  PyramidCylinder volume = new
PyramidCylinder(radius, height);
 }
}
```

## References


[1] Cay Horstmann. (2019). Big Java Early Objects. 7th Ed. Wiley.
[2] Joyce Farrell. (2018). Programming Logic & Design Comprehensive. 9th Ed. Cengage Learning.


## Appendix: Sample of Program Running

